

Candidatos a diagramas UML en OpenMarkov

Informe de la auditoría de Javadoc

Marzo de 2026

Manuel Arias Calleja

Marzo de 2026

Índice

1. Introducción	2
2. Módulo <code>org.openmarkov.core</code>	2
2.1. Modelo de dominio central	2
2.2. Jerarquía de potenciales	2
2.3. Operaciones sobre potenciales	2
2.4. Sistema de ediciones (Undo/Redo)	3
2.5. Restricciones de red	3
2.6. Tipos de red	3
3. Módulo <code>org.openmarkov.inference</code>	3
3.1. Motor de eliminación de variables	3
3.2. Heurísticas de eliminación	3
3.3. Descomposición en DANs simétricos	4
4. Módulo <code>org.openmarkov.gui</code>	4
4.1. Paneles de edición de red	4
4.2. Diálogos de nodo	4
4.3. Componentes de visualización gráfica	4
4.4. Jerarquía de diálogos	4
4.5. Ediciones GUI (Command)	5
4.6. Menús y barras de herramientas	5
4.7. Árbol de decisión visual	5
4.8. Editor de TreeADD	5
5. Módulo <code>org.openmarkov.learning</code>	5
5.1. Framework de aprendizaje	5
5.2. Configuración de algoritmos (GUI)	6
6. Módulo <code>org.openmarkov.io</code>	6
6.1. Lectores y escritores de redes	6
7. Módulos <code>bnEvaluation</code> y <code>sensitivityAnalysis</code>	6
7.1. Framework de evaluación de redes	6
7.2. Análisis de sensibilidad (MVC)	6
8. Módulo <code>org.openmarkov.annotationProcessing</code>	7

8.1. Pipeline de procesamiento de anotaciones	7
9. Resumen	7

1. Introducción

Este documento identifica las clases, paquetes y componentes de OpenMarkov que, por su complejidad estructural, merecen ser acompañados de diagramas UML (de clases, secuencia o paquetes) para facilitar su comprensión.

Los candidatos se han identificado durante una auditoría sistemática de Javadoc que ha recorrido los 19 módulos del proyecto (~1000 ficheros Java). Los criterios para incluir un candidato son:

- Jerarquías de herencia profundas (>5 subclases directas).
- Paquetes con 5 o más clases interrelacionadas.
- Patrones de diseño complejos (Strategy, Composite, Command).
- Clases centrales con muchas dependencias entrantes/salientes.

Para cada candidato se indica el tipo de diagrama recomendado, la ubicación del código y una breve justificación.

2. Módulo `org.openmarkov.core`

2.1. Modelo de dominio central

Paquete: `core.model.network`

Clases clave: `ProbNet`, `Node`, `Variable`, `EvidenceCase`, `Finding`, `State`, `NodeTypeDepot`

Tipo de diagrama: Diagrama de clases (estilo ER).

Justificación: Es el núcleo del dominio. `ProbNet` posee `Nodes` que contienen `Variables` y `Potenciales`; `EvidenceCase` agrupa `Findings`. Un diagrama de las relaciones de composición/agregación facilitaría la comprensión a cualquier nuevo desarrollador.

2.2. Jerarquía de potenciales

Paquete: `core.model.network.potential`

Clases clave: `Potential` (abstracta), `TablePotential`, `UniformPotential`, `ICIPotential`, `GLMPotential`, `FunctionPotential`, `TreeADDPotential`, `ExactDistrPotential`, `SameAsPrevious`, etc.

Tipo de diagrama: Diagrama de clases jerárquico.

Justificación: 30+ subclases de `Potential` organizadas en múltiples niveles de herencia. Incluye el subpaquete `canonical/` (`ICIPotential` → `Max/Min/Tuning`) y `treeadd/` (`TreeADDPotential`, `Branches`, `TreeADDBranch`). Sin un diagrama es difícil entender qué potencial usar en cada situación.

2.3. Operaciones sobre potenciales

Paquete: `core.model.network.potential.operation`

Clases clave: `PotentialOperations`, `DiscretePotentialOperations` y sus clases auxiliares (`Arithmetic`, `Elimination`, `Factory`, `Transform`), más las versiones concurrentes en `concurrent/`.

Tipo de diagrama: Diagrama de colaboración.

Justificación: Las operaciones de multiplicación, marginalización y proyección sobre potenciales están repartidas en múltiples clases estáticas con dependencias cruzadas. Un diagrama mostraría quién llama a quién.

2.4. Sistema de ediciones (Undo/Redo)

Paquetes: `core.action.base` + `core.action.core`

Clases clave: `PNEdit` (abstracta), `CompoundPNEdit`, `PNESupport`, `EditsHistory`, `ConstraintChecker`, y 37+ ediciones concretas (`AddNodeEdit`, `RemoveNodeEdit`, `NodeStateEdit`, etc.)

Tipo de diagrama: (1) Diagrama de clases de la jerarquía. (2) Diagrama de secuencia del flujo `doEdit/undo/redo`.

Justificación: El patrón Command con 37+ comandos concretos es uno de los componentes más complejos del core. `PNESupport` orquesta la ejecución, el historial y los listeners. Sin diagramas, es difícil entender cómo añadir una nueva edición.

2.5. Restricciones de red

Paquete: `core.model.network.constraint`

Clases clave: `PNConstraint` (abstracta), `ConstraintManager`, y 35+ restricciones concretas.

Tipo de diagrama: Diagrama de clases simplificado (lista de restricciones agrupadas por categoría).

Justificación: Muchas restricciones pequeñas con nombres similares. Un diagrama que las agrupe por categoría (topología, tipos de nodo, temporalidad, agentes) evitaría la confusión.

2.6. Tipos de red

Paquete: `core.model.network.type`

Clases clave: `NetworkType` (abstracta) y 12 tipos concretos (`BN`, `ID`, `LIMID`, `DynamicLIMID`, `MDP`, `POMDP`, `DAN`, etc.)

Tipo de diagrama: Diagrama de clases + tabla de restricciones por tipo.

Justificación: Cada tipo de red activa diferentes restricciones. Un diagrama que cruce tipos con restricciones (estilo matriz) sería muy útil como referencia rápida.

3. Módulo `org.openmarkov.inference`

3.1. Motor de eliminación de variables

Paquete: `inference.algorithm.variableElimination`

Clases clave: `VariableElimination` (abstracta), `VariableEliminationCore`, `ChanceVariableElimination`, `DecisionVariableElimination`; tareas: `VEPropagation`, `VEEvaluation`, `VECEAnalysis`; operaciones: `VariableEliminationOperations`

Tipo de diagrama: (1) Diagrama de clases. (2) Diagrama de actividad del bucle de eliminación.

Justificación: Es el algoritmo de inferencia principal. La orquestación entre Core, Chance/DecisionElimination y las tareas concretas no es evidente sin un diagrama.

3.2. Heurísticas de eliminación

Paquete: `inference.heuristic`

Clases clave: `EliminationHeuristic` (abstracta) y 9 implementaciones: `SimpleElimination`, `MinimalFillIn`, `CanoMoralElimination`, `TimeSliceElimination`, etc.

Tipo de diagrama: Diagrama de clases (patrón Strategy).

Justificación: Patrón Strategy clásico con 9 implementaciones. Un diagrama mostraría las alternativas disponibles y sus criterios de selección.

3.3. Descomposición en DANs simétricos

Paquete: `inference.algorithm.decompositionIntoSymmetricDANs`

Subpaquetes: `core/`, `evaluation/`, `ceanalysis/`

Tipo de diagrama: Diagrama de paquetes con dependencias.

Justificación: Paquete complejo con múltiples subpaquetes que implementan la inferencia en redes de análisis de decisión. Un diagrama de paquetes mostraría la organización interna.

4. Módulo `org.openmarkov.gui`

4.1. Paneles de edición de red

Paquete: `gui.window.edition`

Clases clave: `NetworkPanel`, `EditorPanel`, `NetworkEditorPanel`, `EditorInputHandler` y los modos de edición (`EditionMode`, `NodeEditionMode`, `LinkEditionMode`, `SelectionMode`).

Tipo de diagrama: Diagrama de clases + diagrama de estados (modos de edición).

Justificación: La interacción entre paneles, modos de edición e input handlers es el corazón del editor visual. Sin diagrama, es difícil entender dónde añadir un nuevo modo de edición.

4.2. Diálogos de nodo

Paquete: `gui.dialog.node`

Clases clave: `NodePropertiesDialog` y sus paneles internos: `NodeDefinitionPanel`, `NodeDomainValuesTable`, `NodeProbsValuesTablePanel`, `NodeParentsPanel`.

Tipo de diagrama: Diagrama de clases (composición de paneles).

Justificación: El diálogo de propiedades del nodo es la interfaz más compleja del GUI. Compone múltiples paneles que interactúan entre sí.

4.3. Componentes de visualización gráfica

Paquete: `gui.graphic`

Clases clave: `VisualNode`, `VisualLink`, `VisualNetwork`, `InnerBox`, `FSVariableBox`, `NumericVariableBox`.

Tipo de diagrama: Diagrama de clases.

Justificación: Jerarquía de componentes visuales con relaciones de composición (`VisualNode` contiene `InnerBox`, etc.) Esencial para entender el renderizado del grafo.

4.4. Jerarquía de diálogos

Paquete: `gui.dialog.common`

Clases clave: `DialogBase`, `BottomPanelButtonDialog`, `OkCancelDialog`; `PotentialPanel` (abstracta) con 12+ implementaciones por plugin; `RequestDialogger` (builder fluido con generics).

Tipo de diagrama: Diagrama de clases (Template Method + Strategy/Plugin).

Justificación: Toda ventana de diálogo extiende esta cadena de herencia. Los paneles de potenciales se descubren por anotación `@PotentialPanelPlugin` — un diagrama mostraría cómo se conectan.

4.5. Ediciones GUI (Command)

Paquete: `gui.action`

Clases clave: 18 subclases de `PNEdit` (`AddFindingEdit`, `RemoveSelectedEdit`, `PasteEdit`, `ICITablePotentialEdit`, etc.)

Tipo de diagrama: Diagrama de clases.

Justificación: Patrón Command específico de la GUI, complementario al del core (action/core). Incluye ediciones compuestas y relaciones con `PotentialChangeEdit`.

4.6. Menús y barras de herramientas

Paquete: `gui.menutoolbar`

Clases clave: `MenuToolBarBasic` (interfaz), `ContextualMenu`, `MainMenu`, `ToolBarBasic`; `MenuAssistant` (mediador); `ContextualMenuFactory` (factory).

Tipo de diagrama: Diagrama de clases.

Justificación: Múltiples patrones (Factory, Mediator, Plugin vía `@Toolbar`) en un paquete con 22 ficheros.

4.7. Árbol de decisión visual

Paquete: `gui.window.decisiontree`

Clases clave: `DecisionTreeWindow`, `DecisionTreePanel`, `DecisionTreeModel`, `VisualDecisionTree` y paneles de elementos.

Tipo de diagrama: Diagrama de clases (MVC).

Justificación: Subsistema autocontenido de visualización con patrón MVC. Merece un diagrama independiente.

4.8. Editor de TreeADD

Paquete: `gui.dialog.treadd`

Clases clave: `TreeADDEditorPanel`, `TreeADDModel`, `TreeADDPanel`, `TreeADDCellRenderer`.

Tipo de diagrama: Diagrama de clases (MVC).

Justificación: Editor MVC autocontenido para potenciales TreeADD, con 18 ficheros en el paquete.

5. Módulo `org.openmarkov.learning`

5.1. Framework de aprendizaje

Paquetes: `learning.core` + `learning.metric` + `learning.algorithm`

Clases clave: `LearningAlgorithm` (abstracta), `LearningEditProposal`, `ScoreAndPenalize` y métricas (`BayesianMetric`, `K2Metric`, `BDeMetric`, etc.)

Tipo de diagrama: Diagrama de clases inter-módulos.

Justificación: El framework de aprendizaje se reparte en 3 módulos con dependencias cruzadas. Un diagrama global mostraría cómo se conectan algoritmos, métricas y propuestas de edición.

5.2. Configuración de algoritmos (GUI)

Paquete: `learning.gui`

Clases clave: `AlgorithmConfiguration` (anotación), `AlgorithmConfigurationManager` (registro), `AlgorithmParametersDialog` (abstracta) con 8+ implementaciones concretas.

Tipo de diagrama: Diagrama de clases (Strategy/Registry).

Justificación: Cada algoritmo de aprendizaje tiene su propio diálogo de parámetros, descubierto por anotación.

6. Módulo `org.openmarkov.io`

6.1. Lectores y escritores de redes

Paquetes: `io` + `io.probmodel.reader` + `io.probmodel.writer`

Clases clave: `ProbNetReader` / `ProbNetWriter` (interfaces), `PGMXReader_0_2`, `PGMXWriter_0_2`, y los formatos alternativos en `io.database.elvira`, `io.database.weka`, `io.database.excel`.

Tipo de diagrama: Diagrama de clases (patrón Strategy/Factory para formatos).

Justificación: Múltiples formatos de archivo con interfaces comunes. Un diagrama mostraría qué formatos existen y cómo se descubren via `@ProbNetFormatType`.

7. Módulos `bnEvaluation` y `sensitivityAnalysis`

7.1. Framework de evaluación de redes

Paquete: `bnEvaluation`

Clases clave: `NetEvaluator`, `LearningEvaluator`, `SplitSetManager`, `SplitSet`; jerarquía de medidas: `Measure` → `MeasureValue`, `MeasureMatrix`, `MeasuresSet`.

Tipo de diagrama: (1) Diagrama de clases de la jerarquía de medidas. (2) Diagrama de actividad del pipeline (`split` → `evaluate` → `aggregate`).

Justificación: Pipeline de evaluación con múltiples clases de datos (medidas, matrices, conjuntos) que se acumulan y promedian.

7.2. Análisis de sensibilidad (MVC)

Paquete: `sensitivityAnalysis`

Clases clave: `SensitivityAnalysisModel` (observable), `SensitivityAnalysisController`, `SensitivityAnalysis`. 6+ diálogos concretos según `AnalysisType`.

Tipo de diagrama: Diagrama de clases (MVC).

Justificación: Patrón MVC explícito con modelo observable y múltiples vistas especializadas.

8. Módulo `org.openmarkov.annotationProcessing`

8.1. Pipeline de procesamiento de anotaciones

Paquete: `annotationProcessing.localization_bindings`

Clases clave: `BindLocalizations` (anotación), `BindLocalizationsProcessor` (APT), `XMLConstantsParser`, `ClassDefinition` (record).

Tipo de diagrama: Diagrama de actividad.

Justificación: Pipeline de generación de código en tiempo de compilación: anotación → procesador → parser XML → árbol de clases → archivos generados.

9. Resumen

#	Módulo	Componente	Tipo de diagrama
1	core	Modelo de dominio (ProbNet, Node, Variable)	Clases (ER)
2	core	Jerarquía de potenciales (30+)	Clases jerárquico
3	core	Operaciones sobre potenciales	Colaboración
4	core	Sistema de ediciones (Undo/-Redo)	Clases + secuencia
5	core	Restricciones de red (35+)	Clases agrupado
6	core	Tipos de red (12)	Clases + matriz
7	inference	Eliminación de variables	Clases + actividad
8	inference	Heurísticas (Strategy)	Clases
9	inference	Descomposición en DANs	Paquetes
10	gui	Paneles de edición de red	Clases + estados
11	gui	Diálogos de nodo	Composición
12	gui	Componentes gráficos	Clases
13	gui	Jerarquía de diálogos	Clases (Template)
14	gui	Ediciones GUI (Command)	Clases
15	gui	Menús y toolbars	Clases
16	gui	Árbol de decisión visual	Clases (MVC)
17	gui	Editor de TreeADD	Clases (MVC)
18	learning	Framework (3 módulos)	Clases inter-módulos
19	learning	Config. de algoritmos GUI	Clases (Strategy)
20	io	Lectores/escritores de redes	Clases (Strategy)
21	bnEvaluation	Medidas y pipeline	Clases + actividad
22	sensitivityAn.	MVC de análisis	Clases (MVC)
23	annotationProc.	Pipeline de anotaciones	Actividad

Total: 23 diagramas recomendados, distribuidos en 8 módulos.

Prioridad alta (componentes más referenciados y complejos): números 1 (modelo de dominio), 2 (potenciales), 4 (sistema de ediciones) y 7 (eliminación de variables).

Prioridad media: 10–12 (editor visual), 13 (jerarquía de diálogos), 18 (learning) y 20 (I/O).

Prioridad baja: el resto, útiles para documentación completa pero menos críticos para el desarrollo diario.